A guide to better writing and reporting

Nan

Table of contents

1 Preface 2

2 Writing 2

3 Plotting 3

4 Coding with AI 3

5 Steps for processing feedback 4

6 Reduce information leakage 4

1 Preface

An experiment report is where thinking becomes visible. Yet reports often drift away from their purpose: figures accumulate without an argument to hold them together, feedback is read once and half-forgotten, and code or prose borrowed from an AI assistant enters the document faster than it enters our understanding. The result is writing that is longer but not clearer, and analyses that look finished but cannot be defended.

This guide collects the principles the author has found most useful for pushing back against that drift. It is written in the first person, as a set of working habits rather than rules, and it is meant to be reread before starting a report and again before sending one out. The living version is generated from https://github.com/nanwanglin/templates/blob/master/experiment-report.qmd, so it can be updated as the habits sharpen.

The chapters move from the page to the process: how to write and plot so that a reader can follow the reasoning, how to collaborate with AI without outsourcing understanding, and how to absorb feedback so that it changes the work rather than just the surface. A few principles run through all of them.

The general principles are:

- Organize a report around its key findings, not around a laundry list of figures. Every figure should earn its place by supporting a claim the reader can state in one sentence.
- Name section headings as questions whenever possible. A question ("Does adaptation persist across talkers?") tells the reader what the section is trying to resolve; a noun phrase ("Adaptation results") only tells them the topic.
- Pause deliberately before and after implementation, rather than moving straight from feedback to keystrokes.
  - Pause *before* implementing to think clearly about: (1) what the goal is; (2) what steps are necessary to reach it; (3) what the gist of the feedback is, and what motivation lies behind the suggested action; (4) which components matter most; and (5) what my tentative plan is, and whether AI has suggestions once it sees that plan.
  - Pause *after* implementing to ask: (1) whether the writing is aligned with the principles in this document; (2) how to summarize the observed patterns and what underlying factors might explain them; (3) what to explore next; and (4) what help I need from others, and from whom.
- Limit copy-pasting AI-generated text or code *without understanding it*. Unexamined material confuses the reader and, worse, hides the gaps in my own understanding from me. If I cannot explain a line, it does not belong in the report yet.

2 Writing

Good writing in a report is not decoration; it is the shortest path between what I found and what the reader understands. Three demands are worth holding in tension: informativeness, clarity, and correctness.

- Write with maximum informativeness. Prefer statements that a reader can act on or check. Instead of "the parameter was estimated," say what was estimated and in what sense — for

example, whether a value is a prior or a posterior, and of which quantity. Vague nouns ("the effect," "the model," "the result") should be pinned down the first time they appear.

- Write with maximum clarity.
  - Structure is the main lever for clarity. Put yourself in the reader's position and build each idea up from what they already know before introducing what they do not. When describing a figure, start from its overall purpose, then name the axes, and only then walk through the lines or points — never the reverse.
  - Prioritize conceptual understanding over code description. Readers need to know *what* an analysis does and *why*, not a narration of the function calls that produced it. Keep implementation detail in the code or an appendix.
  - Avoid private jargon, informal shorthand that only makes sense to me. If a term is genuinely useful, define it once, plainly, and then use it consistently.
- Write with correct grammar and notation.
  - Use complete sentences and fix grammatical errors; a sentence the reader has to reparse costs them attention that belongs to the science.
  - Avoid spelling errors, and use mathematical notation precisely: match symbols to their definitions, keep subscripts consistent, and do not reuse one letter for two quantities.

3 Plotting

A figure is an argument made visually. It should make the intended comparison easy and every other comparison secondary.

- Use consistent coloring, theme, and capitalization for the same variables or conditions throughout the document.
- Write stand-alone captions. A caption should let a reader understand the figure without hunting through the surrounding text: state what is plotted, on what data, and what the reader should take away.
- Keep as much of the raw information as the plot can bear — individual points, distributions, or trajectories — rather than collapsing everything to a mean that hides the variation behind it.
- Remove figure junk. Delete gridlines, legends, borders, and annotations that do not serve the comparison; each removed distraction makes the real signal easier to see.
- Plot uncertainty around estimated parameters whenever possible (for example, bootstrapped or posterior intervals), so the reader can see how much to trust each estimate rather than reading a point as if it were exact.
- Plot a baseline or reference whenever possible — chance level, a null model, a control condition — so that the size of an effect can be judged against something, not admired in isolation.

4 Coding with AI

- Before writing any code, think clearly about the key behavioral signature I expect, the hypotheses being tested, and the sanity checks that would catch a mistake — for instance, visualizing parameters or simulating from the model to confirm it behaves as intended.
- Work on the code outline first: name the inputs, the transformations, and the outputs, so that the structure is decided before the syntax.

- Early on, hand the AI a simplified version of the problem and ask it to verify and improve it. Small, checkable pieces let me confirm the logic before scale hides errors.
- Only afterwards hand over the fuller implementation to AI — and read what comes back closely enough to explain it, so that acceleration does not become a loss of understanding.

5 Steps for processing feedback

- Collect all feedback into a single document, so that comments from different people, meetings, and channels can be seen and prioritized together.
- Read through everything once to form a general impression before acting, and identify the few most important action items rather than starting with whatever comment is first.
- Implement the feedback together with AI where that helps, keeping the earlier principle in mind: understand each change well enough to defend it.
- Think through *why* each piece of feedback was given — the motivation behind the comment, not just its literal instruction — so that the fix addresses the underlying concern rather than the symptom.
- Double-check that every item has actually been implemented, then write a short email back to the reviewer summarizing what was changed and, briefly, what was not and why.

6 Reduce information leakage

"Information leakage" here means the quiet gap between feeling that I understand something and actually being able to reconstruct it. Closing that gap is what turns feedback and AI output into real understanding.

- Ask more about the motivation behind a suggested action item. Knowing *why* a change was requested lets me generalize it to the next case instead of applying it blindly to this one.
- Watch for vague, partial understanding — the state of "this feedback or AI suggestion seems to make sense, though I could not have generated it myself." That feeling is a signal to slow down and reconstruct the reasoning until I could.
- Prepare my mind before any meeting and make sure it is in a clear state; understanding formed while distracted tends to leak out afterward.
- Seek written feedback over oral feedback where possible. Written comments can be reread, quoted exactly, and tracked to completion, whereas spoken feedback is easy to half-remember and hard to verify later.